

Arquitectura de Computadoras — Proyecto 2

Parte 1

Diseño e implementación de la extensión 'A' (atómica) de RISC-V en un procesador pipelined

“Don’t hurry your code. Make sure it works well and is well designed.”

— Linus Torvalds

1 Introducción y objetivos

En la actualidad, la computación multi-core depende de primitivas de sincronización para coordinar accesos compartidos a memoria entre distintos hilos de ejecución. En hardware, estas primitivas se implementan mediante **instrucciones atómicas especialmente diseñadas para garantizar exclusión mutua entre procesos y evitar condiciones de carrera** (*race conditions*). Muchos lenguajes de programación modernos integran estas primitivas, como C++ mediante sus clases `std::atomic` y `std::mutex`.

El objetivo de este proyecto es implementar la **extensión 'A' de RISC-V para instrucciones atómicas** dentro de un procesador pipelined base. Los estudiantes analizarán el impacto a nivel microarquitectural de secuencias escritura-modificación-lectura (*read-modify-write*) en el hazard unit, diseñarán interbloqueos de sincronización adecuados, y expandirán el control y datapath del procesador para lograr ejecución atómica correcta.

Este enunciado corresponde a la primera parte del proyecto, en la cual se implementará el procesador pipelined base sobre el que se implementará la extensión 'A'.

2 Requerimientos: Procesador pipelined base

Se debe **implementar el procesador RISC-V pipelined visto en clase** con las instrucciones especificadas en el section [A](#). Como recordatorio, este pipeline debe tener las siguientes 5 etapas:

1. **Instruction Fetch (IF):** Generación del *program counter* (PC) y acceso al *instruction memory*.
2. **Instruction Decode (ID):** Lectura del *register file*, extensión de *immediate* y generación de señales de control.
3. **Execute (EX):** Cómputos con el ALU y resolución de *branches*.
4. **Memory Access (MEM):** Escrituras y lecturas a la *data memory*.
5. **Write Back (WB):** Escritura de resultados al register file.

Como se ha visto en clase, este procesador debe tener un **hazard unit** robusto con las siguientes capacidades:

- **Forwarding:** Enviar valores de resultado de MEM/WB a EX directamente (haciendo bypass de los valores leídos en ID) para resolver dependencias RAW (*read-after-write*) entre registros.
- **Stalling:** Detener temporalmente IF ante dependencias RAW entre la memoria y un registro para dar suficiente tiempo a la lectura de memoria.

- **Flushing:** Descartar las instrucciones en IF y ID del pipeline al hacer un *jump* o un *branch*, ya que estas corresponden a código ejecutado si no se hubiera realizado el salto.

El procesador debe ser probado con testbenches y programas que manifiesten los posibles conflictos que el hazard unit resuelve. Los testbenches deben mostrar el flujo de señales del pipeline y el funcionamiento correcto del hazard unit.

3 Entregables para la parte 1

1. **Diseño en Verilog:** Un archivo .zip con el código fuente del proyecto, programas de Assembly utilizados y testbenches.
2. **Informe:** Informe con detalles del diseño e implementación, y análisis de waveforms comprobando la correctitud del procesador y el hazard unit bajo distintos casos de conflictos. Este informe será extendido en la parte 2.

4 Referencias

- Portal de especificaciones de RISC-V oficial:
<https://riscv.org/technical/specifications/>
- Especificación del ISA no privilegiado de RISC-V:
https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf

A Instrucciones base a implementar

Instrucción	Descripción
lw rd, imm(rs1)	Load word
addi rd, rs1, imm	Add immediate
slli rd, rs1, shamt	Shift left logical immediate
xori rd, rs1, imm	XOR immediate
srli rd, rs1, shamt	Shift right logical immediate
srai rd, rs1, shamt	Shift right arithmetic immediate
ori rd, rs1, imm	OR immediate
andi rd, rs1, imm	AND immediate
sw rs2, imm(rs1)	Store word
add rd, rs1, rs2	Add
sub rd, rs1, rs2	Subtract
sll rd, rs1, rs2	Shift left logical
xor rd, rs1, rs2	XOR
srl rd, rs1, rs2	Shift right logical
sra rd, rs1, rs2	Shift right arithmetic
or rd, rs1, rs2	OR
and rd, rs1, rs2	AND
lui rd, imm	Load upper immediate
beq rs1, rs2, offset	Branch if equal
bne rs1, rs2, offset	Branch if not equal
blt rs1, rs2, offset	Branch if less than
bge rs1, rs2, offset	Branch if greater than or equal
jalr rd, rs1, imm	Jump and link register
jal rd, offset	Jump and link

Cuadro 1: Instrucciones a implementar en el pipeline base.